

Project Title:

Forecasting Household Energy Consumption Using Temporal Feature Engineering and Gradient Boosting

Problem Definition

Household electricity consumption is highly irregular and nonlinear, driven by a mix of daily routines, weekly patterns, seasonal weather effects, and unpredictable appliance usage. This variability makes it difficult for utilities and consumers to anticipate demand, plan capacity, or manage costs.

The goal of this project is to forecast household energy consumption (Global Active Power, measured in Wh) using historical minute-level measurements from 2006 to 2010. The core challenge is to capture the underlying temporal structure, daily, weekly, and yearly cycles, while preventing data leakage and overfitting in a dataset of over 2 million observations.

The problem is framed as a supervised regression task:

- Input (features):** Time-based features (hour, day, week, month, season), Fourier terms for smooth seasonality, and lagged values of the target variable.
- Output (target):** Energy consumption at a given minute.
- Evaluation:** Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE) on unseen future data, using time-series cross-validation.

Dataset:

Dataset Overview

This study uses a household energy consumption dataset that recorded measurements at one-minute intervals between 2006 and 2010. The dataset is publicly available at the following link: [individual+household+electric+power+consumption](#)

It contains the following features:

- Global Active Power (kW)
- Global Reactive Power (kVAR)
- Voltage (V)
- Global Intensity (A)
- Sub-metering 1 (Wh)
- Sub-metering 2 (Wh)
- Sub-metering 3 (Wh)

After loading the dataset, an initial overview is performed. This overview summarizes the data types, memory usage, descriptive statistics for both numerical and categorical features, and the number of missing values in each column.

The missing-value counts indicate that all energy-related features share the same number of NaN values, suggesting the gaps occur at identical timestamps across columns. To handle these missing values, their exact locations and durations will be examined in subsequent steps.

Dataset Explanation

Global Active Power (kW)

The total real power consumed by the household; the power that actually performs useful work (lighting, heating, running motors) and is billed by the utility. It is a rate of energy use, measured in kilowatts (kW). To convert it into energy (Wh or kWh), it is multiplied by the time interval:

$$\text{Energy (Wh)} = \text{Global Active Power (kW)} \times 1000 / 60$$
for a 1-minute sampling interval.

Global Reactive Power (kVAR)

The portion of power that does not perform useful work but is required to sustain the magnetic fields in motors, transformers, and inductive loads. It is measured in kilovolt-amperes reactive (kVAR) and is not billed directly, but it affects the overall efficiency and quality of the electrical supply.

Sub-metering 1, 2, 3 (Wh)

Energy consumption (in watt-hours) recorded by three dedicated sub-meters, each monitoring a specific circuit group:

- Sub-metering-1: Kitchen (dishwasher, oven, microwave)
 - Sub-metering-2: Laundry room (washing machine, dryer, refrigerator, light)
 - Sub-metering-3: Electric water heater and air conditioner
- These sub-meters cover only a fraction of total household consumption; the remainder is captured by the difference between Global Active Power and the sum of the three sub-meters.

Voltage (V)

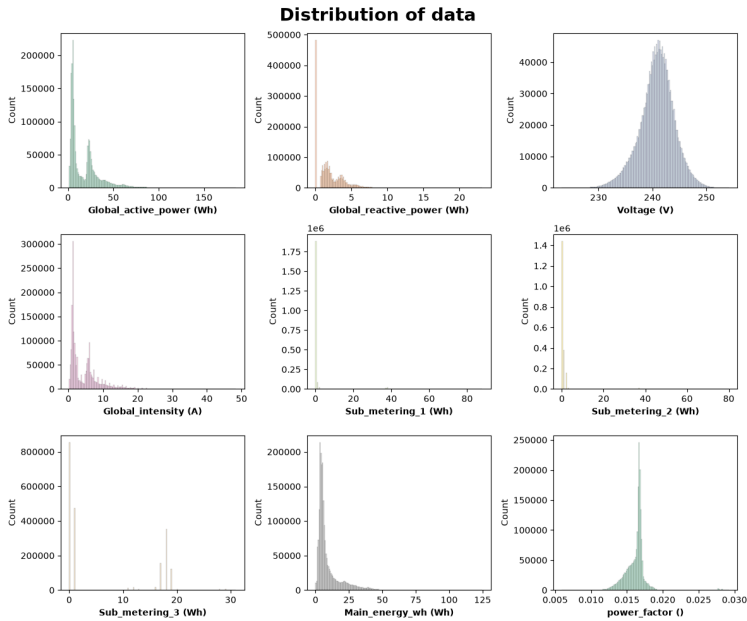
The electrical potential difference that drives current through the circuit; the push that moves electrons. In residential settings it is typically 120 V (North America) or $230\text{-}240\text{ V}$ (Europe). Voltage in this dataset is measured in volts (V) and usually fluctuates within a narrow band around its nominal value.

Global Intensity / Current (A)

The rate of electric charge flow through the main circuit, measured in amperes (A). It reflects how much electricity is being drawn at any instant; more appliances running means higher current. It is related to power and voltage by:

$$P = V \times I$$

where P is power (watts), V is voltage (volts), and I is current (amperes).



`Main-energy-wh` is a new feature which will be explained in the following part.

Almost most of the features and the target are left-skewed; the bulk of the data is concentrated toward the low end of the scale, with a long tail extending toward higher values. This reflects the nature of household consumption: most minutes the household is idle or drawing minimal power, while occasional appliance usage produces the high-value tail.

The original `Global_active_power` exhibits a bimodal distribution with two peaks, one for idle periods (near-zero consumption) and one for active periods (moderate appliance usage). After subtracting the `sub-meterings`, the resulting `Main_energy_wh` exhibits a single-peaked, left-skewed distribution. This change occurs because removing the concentrated high-power events (water heating, AC, laundry) eliminates the second mode, leaving only the diffuse low-level background signal from many small appliances.

Feature Engineering and Data Preprocessing

Feature Engineering

Unit Conversion

The units are converted from kilowatts (`kw`) to watt-hours (`wh`). By combining the date and time columns into a single datetime index, further data analysis is simplified.

Conceptual Features

1. **Main Energy (`wh`)** = `Global-Active-Power` - Σ (`Sub-meterings`)

This represents the energy consumed by all other appliances not covered by the three sub-meters. It constitutes the largest share of household consumption. Negative values are removed by clipping: `df[a] = df[a].clip(lower=0)`

Note: The `sub-meterings` sum to roughly half of the global consumption. The remaining portion ("Main energy") covers everything else: lighting, electronics, and miscellaneous appliances.

2. **Apparent Power (`va`)** = `V` × `I`

The total power the grid has to deliver to your house. It includes:

- The useful part (what actually runs your appliances)
- The reactive part (what charges coils, motors, transformers)

3. **Active Power (`w`)** = `V` × `I` × **power factor**

The useful part of the apparent power; the power that actually does work (heats, spins, lights) and that you're billed for. Power factor tells you what fraction of the apparent power is useful:

- Power factor = 1.0 → everything is useful (ideal case, pure resistive loads)
- Power factor = 0.8 → only 80% is useful (typical when motors are running)
- Power factor < 0.8 → lots of waste (heavy inductive load)

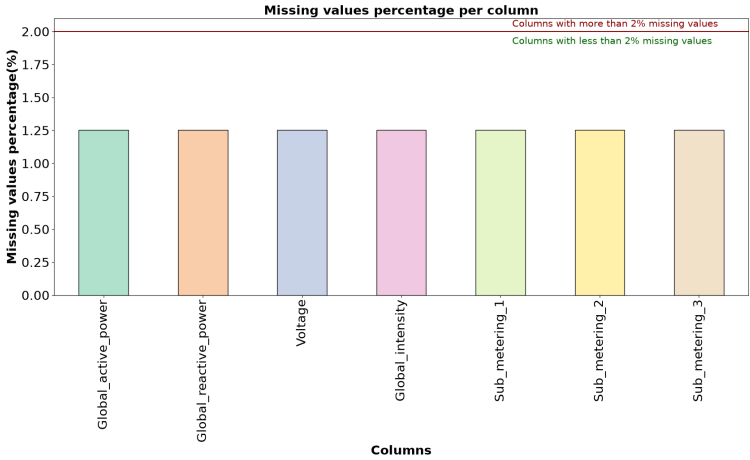
Temporal Features:

Three categories of temporal features were engineered to capture the time-dependent structure of the data:

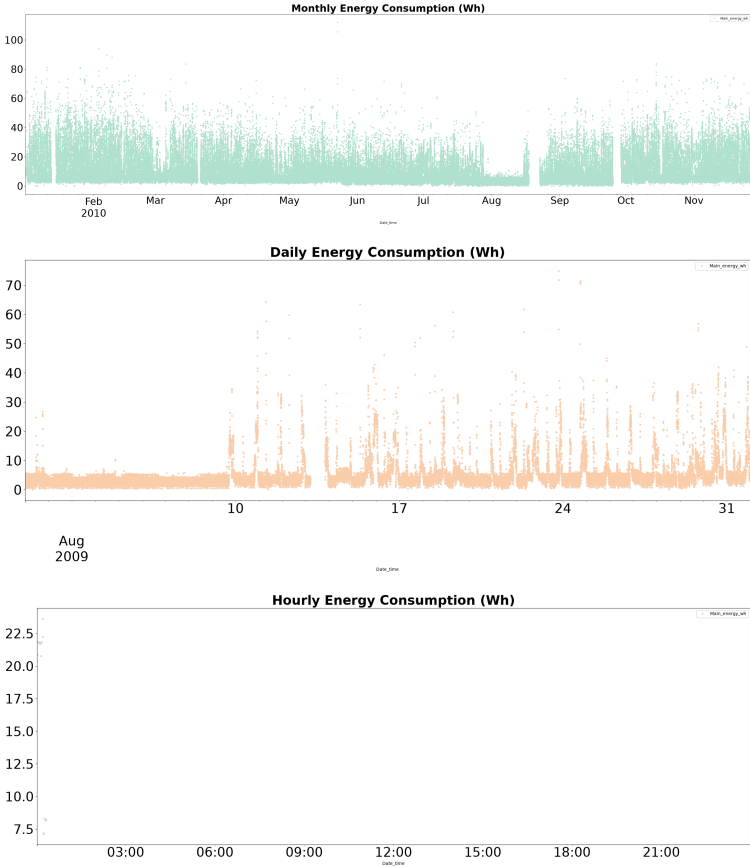
- **Fourier Features:** model smooth, repeating patterns (trend and seasonality) using sine and cosine waves at multiple frequencies.
- **Time Series Features:** extract calendar-based indicators (hour, day of week, month, quarter, season) to help the model distinguish between seasonal periods.
- **Lag Features:** use past values of the target variable to capture autocorrelation and recurring cycles (e.g., yearly patterns).

Data Processing

NaN Values Mitigation



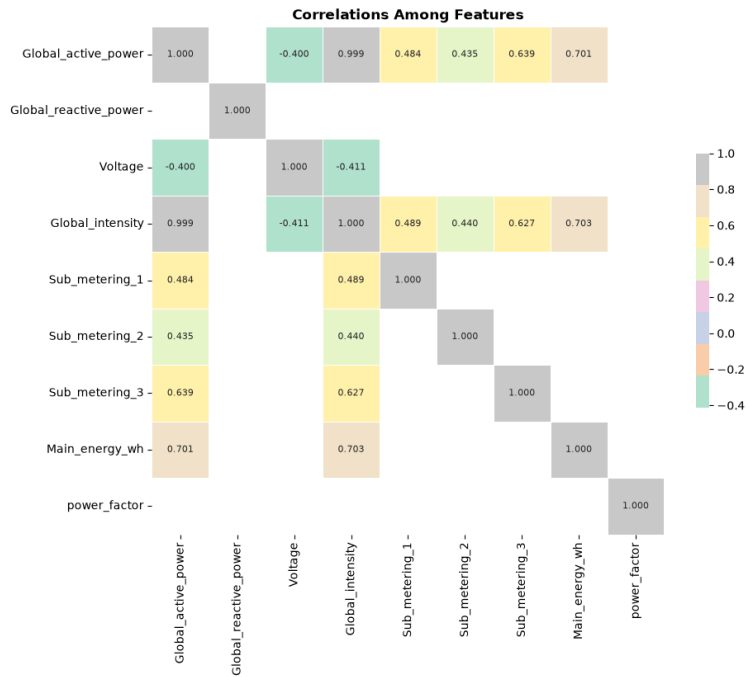
The percentage of missing values is less than 1.25% and is equal across all features, indicating that gaps occur at the same timestamps.



These figures show the missing values in monthly, daily and hourly time periods. Data analysis revealed that the gaps are all short (1-5 days) and scattered across different years and seasons. Since these gaps are small relative to the dataset size, interpolation was used to fill them using neighboring values, preserving the continuity of the time index.

Data Analysis

Linear Correlation



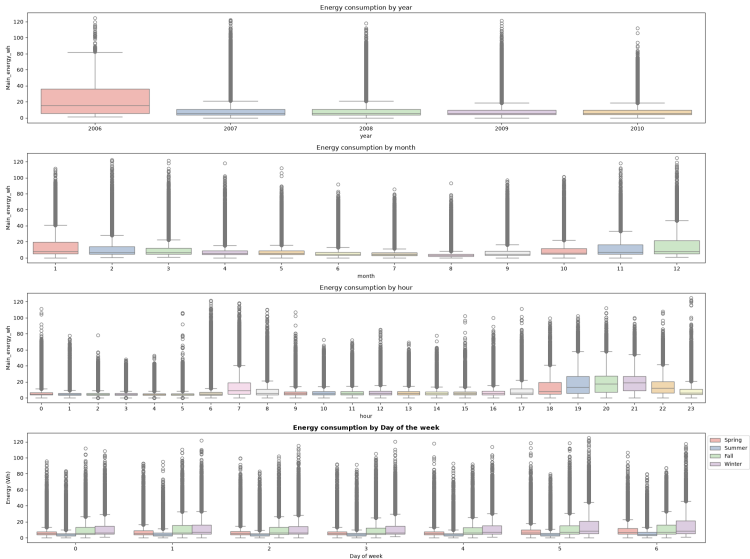
The correlation analysis is valuable not because it identifies features to include, but because it identifies features to exclude. Global Intensity (0.99) and Main_energy_wh (0.70) are strongly correlated with the target only because they are essentially the target in disguise. They must be dropped or lagged to avoid leakage.

Temporal features, calendar indicators, Fourier terms, and lagged targets, are superior for forecasting because they are known in advance, capture human behavioral patterns, and generalize to unseen future data. This is why the project's feature engineering focused on the temporal dimension rather than the physical-electrical features.

Consumption Distribution Across Time Scales

To analyze how household energy consumption varies across different temporal dimensions (year, month, hour, and day of week), a 4-panel boxplot figure is generated, with each panel representing a different time-based grouping:

- **Year:** It shows whether consumption levels differ across years (long-term stability or drift). Except 2006, median consumption is similar across years, confirming the stationary behavior observed in the trend analysis.
- **Month:** It shows seasonal patterns. There is clear seasonal variation; consumption peaks in fall and winter, dips in summer.
- **Hour:** It shows daily usage profile. There is a distinct daily profile; low overnight, rising in the morning, peaking in the evening.
- **Day of week × season:** It shows weekly patterns broken down by season. Weekends demonstrate more consumption than weekdays and consumption patterns across seasons are similar through days of the week.



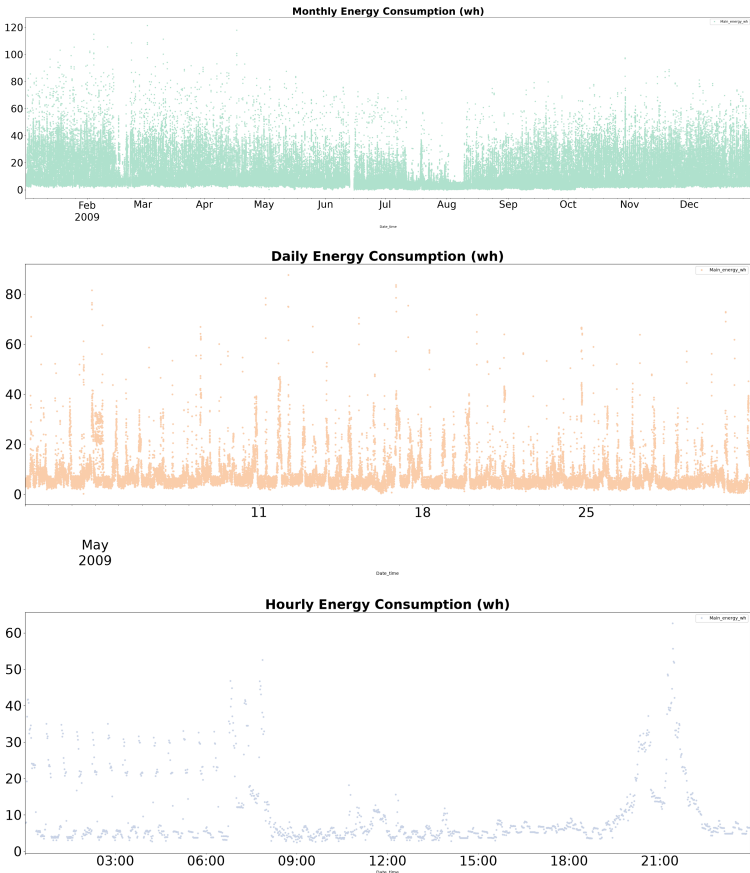
This analysis validates the choice of temporal features for modeling:

- The strong hourly and monthly patterns justify including `hour` and `month` as features.
- The seasonal variation supports using Fourier features to capture smooth annual cycles.
- The weekly differences confirm the importance of `dayofweek` as a predictive feature.

Consumption Patterns Across Different Time Scales

Visualizing the raw energy consumption signal at three nested time scales (one year, one month, and one day) in order to identify recurring patterns at each level. Three separate plots are generated, each zooming into a smaller time window:

- **Monthly view:** Consumption shows clear seasonal structure, with elevated usage during fall and winter months and reduced usage in spring and summer. This justifies Fourier features for smooth seasonality.
- **Daily view:** A repeating daily pattern in month is visible. This confirms that `dayofmonth` is not a meaningful predictive feature.
- **Hourly view:** A distinct daily cycle appears; consumption rises in the morning, peaks in the evening, and drops overnight. This validates `hour` as a key feature.

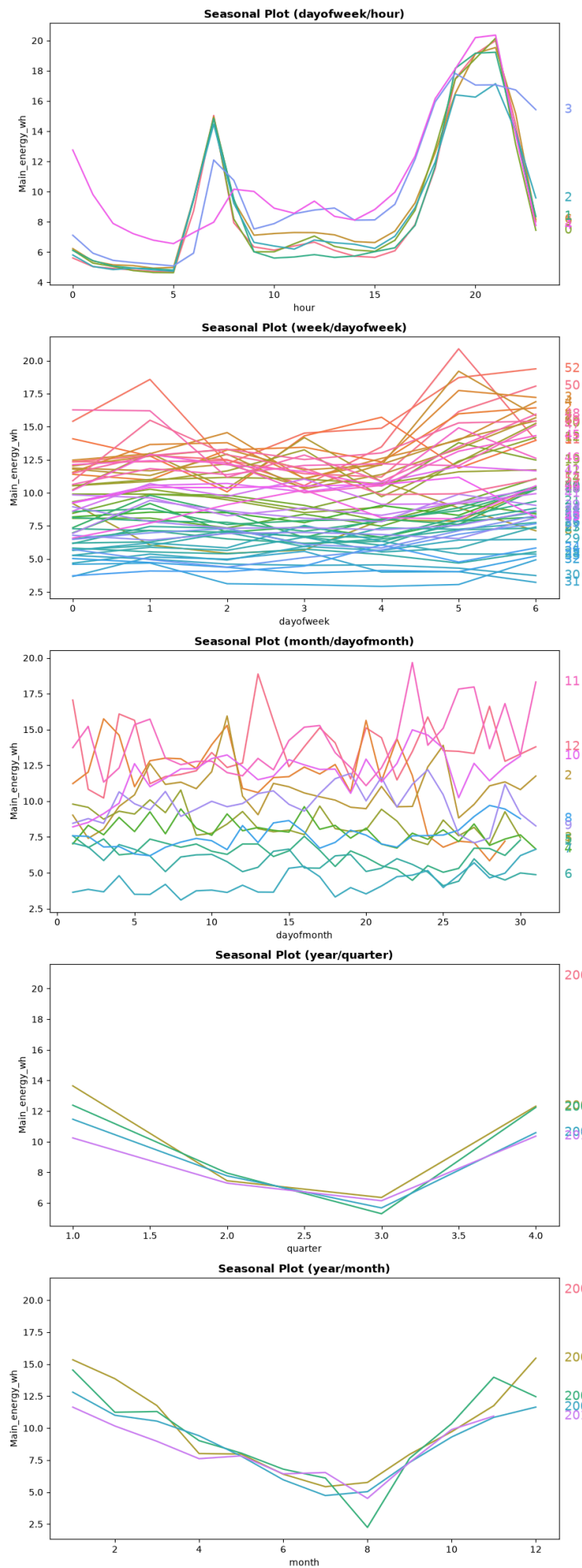


This nested visualization confirms that energy consumption contains structure at multiple time scales simultaneously. No single time resolution captures the full picture:

- A model trained only on hourly patterns would miss seasonality.
- A model trained only on yearly patterns would miss daily behavior.

Together, these plots justify the multi-scale feature engineering strategy used in this project: Fourier features for annual cycles, calendar features for weekly patterns, and hour-based features for daily behavior.

Seasonal Pattern



Hourly pattern (visible across all days of the week): Consumption peaks during the evening and early morning hours, reflecting typical daily routines such as waking up, cooking, and returning home from work.

Weekly pattern: Consumption is consistently higher on weekends and lower on weekdays, indicating that occupants spend more time at home and use more appliances during weekends.

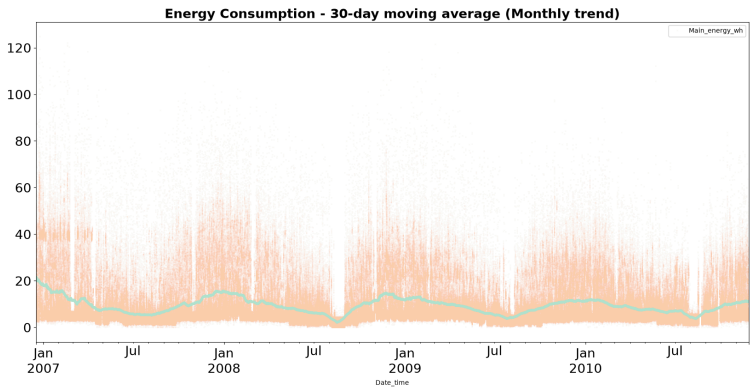
Day of month: The distribution of consumption across different days of the month shows no consistent or meaningful pattern, confirming that `dayofmonth` is not an informative feature for modeling.

Quarterly pattern: Across all years, Q3 (July–September) shows the lowest consumption, while Q1 (January–March) and Q4 (October–December) show the highest. This reflects the combined effects of seasonal heating demand in winter and reduced activity in summer.

Monthly pattern: Consumption reaches its minimum in August across all years, while peaks occur during the winter months, consistent with heating demand and increased indoor activity during colder periods.

Moving Average:

To reveal the underlying trend in household energy consumption by smoothing out short-term noise from the raw minute-level signal, a 30-day moving average is computed on the `Main_energy_wh` column. For each timestamp, the average of all values within a centered 30-day window is calculated, requiring at least half the window (15 days) to contain valid data. This produces a smoothed curve representing the monthly trend while preserving the overall shape of consumption over time.



The results reveal:

- The 30-day window is short enough to preserve seasonal patterns (winter peaks, summer dips) but long enough to remove daily noise.
- Rapid spikes (single-day anomalies) are visually flattened, making the long-term movement easier to identify.
- The trend line reveals whether consumption is gradually rising, falling, or stable across the 4-year period.

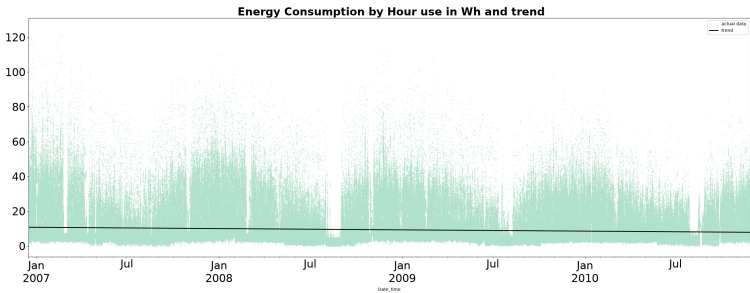
This analysis confirms that the data contains meaningful seasonal structure (not just random noise), which justifies the use of Fourier features and calendar-based features in the forecasting model.

Trend:

To quantify the overall direction of energy consumption across the full time span a simple linear regression is fitted with:

- **Feature:** `trend` : a sequential time counter (0, 1, 2, ..., n)
 - **Target:** `Main_energy_wh` : the energy consumption per minute
- The model learns the equation: $\text{Energy} = w \cdot \text{time} + b$
Where w is the slope (rate of change per minute) and b is the intercept.

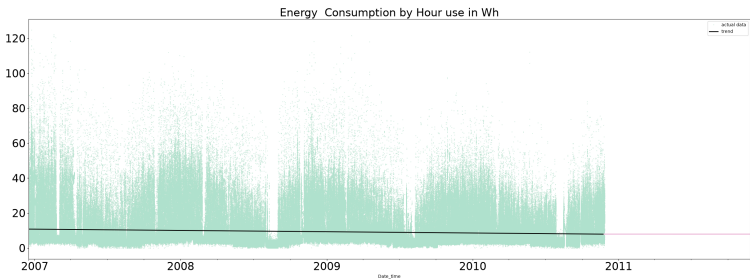
The slope coefficient is approximately 10^{-6} , indicating an essentially flat trend. Over the entire 4-year period, the total change in consumption is negligible; the data is stationary around a constant mean.



Results show:

- No meaningful long-term increase or decrease in household consumption.
- The visible fluctuations in the raw data are driven by seasonality and daily patterns, not by a growing or shrinking baseline.
- Therefore, detrending is unnecessary; subtracting a trend line would have almost no effect on the data.

A future trend prediction (pink line) is also generated for the year 2010-2011 by extending the `trend` feature. Because the slope is near zero, the future trend line remains essentially flat; reinforcing that no long-term drift exists.



Since no meaningful trend is found, mean-centering is chosen as the appropriate preprocessing step.

Centered Value = Original Value - Mean

The centered data oscillates around zero instead of around the original mean. This transformation:

- Removes the baseline offset so that positive values represent above-average consumption and negative values represent below-average consumption.
- Improves numerical stability for models that assume zero-centered targets (e.g., linear models, neural networks).
- Does not distort the shape of seasonal or daily patterns; only the vertical position of the data shifts.

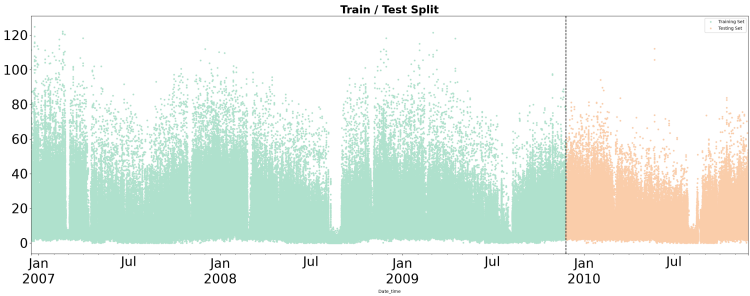
Model building:

Dataset Splitting for Model Building

Train/Test Split:

The dataset is divided into training and testing sets using a temporal cutoff at 26 November 2009. All observations before this date are assigned to the training set, and all observations from this date onward formed the test set. This split preserves the natural time order of the data and simulates a realistic forecasting scenario; training on the past and predicting the future.

The split is visualized to confirm that both sets are contiguous in time, with a vertical dashed line marking the boundary between them.



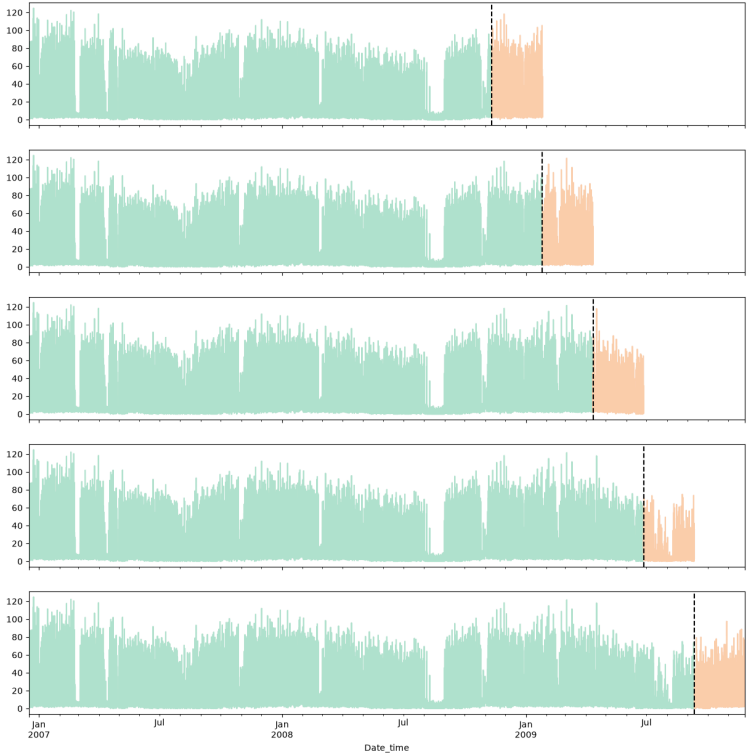
Time Series Cross-Validation:

Within the training set, a 5-fold `TimeSeriesSplit` is applied to generate multiple train/evaluation pairs for robust model validation. The parameters are configured as:

- `n_splits=5` : five sequential folds
- `test_size=5*60*365` : each evaluation window spans one year of minute-level data
- `gap=5*60` : a 5-minute gap between training and evaluation data to prevent any temporal leakage

Unlike standard k-fold cross-validation (which shuffles data randomly), `TimeSeriesSplit` respects chronological order, each fold trains on earlier data and evaluates on later data. This design mirrors real-world forecasting conditions and prevents the model from "seeing the future" during training.

The five folds is visualized in a stacked plot, with training and testing segments distinguished by color and the fold boundary marked with a vertical line. This confirms that each fold provides a distinct, non-overlapping evaluation window, allowing the model's performance to be assessed across multiple years and seasonal conditions.



Linear Regression Model

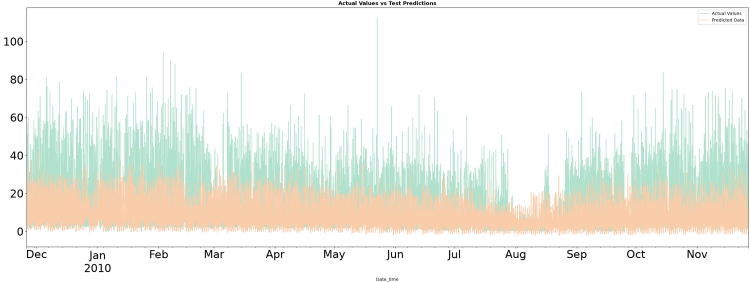
A Linear Regression model is trained as a baseline to establish a reference performance level before applying more complex models like `XGBoost`. The goal is to see how well a simple linear combination of temporal features can forecast energy consumption.

The model is fitted on the training set using the engineered feature set (`features_linear_reg`) which includes:

- Trend and Fourier terms (for seasonality)
- Calendar features (`hour` , `month` , `year` , `dayofweek` , `dayofyear` , `weekofyear`)
- Lag features (`lag1year` , `lag2year` , `lag3year`)

The target is `Main_energy_wh`.

Predictions is generated on both the training set (to check fit) and the test set (to check generalization). The test-set predictions are visualized against the actual values to assess how well the model captures the underlying pattern.



Two metrics are used to quantify performance:

- MAE (Mean Absolute Error): Average absolute error in `Wh` ; interpretable as on average, predictions are off by `X Wh`
 - RMSE (Root Mean Squared Error): Penalizes large errors more heavily; useful for detecting whether the model makes occasional large mistakes
- The linear regression model achieved a Mean Absolute Error (MAE) of 4.63 `Wh` and a Root Mean Squared Error (RMSE) of 7.00 `Wh` on the test set. The MAE indicates that, on average, each prediction deviates from the actual consumption by approximately 4.6 watt-hours. The higher RMSE reflects the presence of occasional larger errors, most likely during sudden consumption spikes that a linear model cannot capture. This baseline establishes the reference performance against which the `XGBoost` model is compared.

Since the gap between MAE and RMSE suggests the presence of nonlinear error patterns, a tree-based model (`XGBoost`) was subsequently trained to determine whether nonlinear feature interactions could reduce these larger errors.

XGBoost Model

Initial XGBoost Model Training

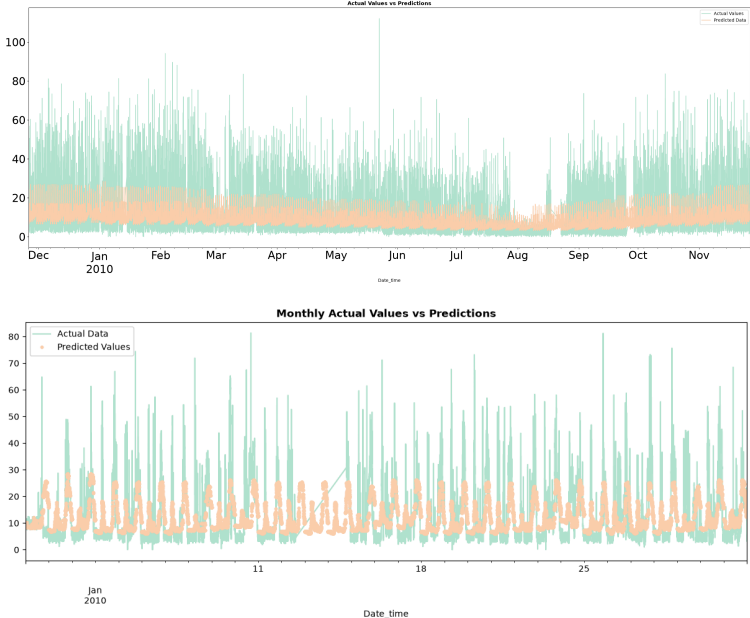
The XGBoost regressor is evaluated using 5-fold TimeSeriesSplit cross-validation with a one-year evaluation window per fold. Individual fold RMSE scores ranged from 4.15 Wh (Fold 2) to 7.59 Wh (Fold 5), with an average of approximately 5.70 Wh. This variability reflects the natural year-to-year differences in household consumption patterns, some years are more regular and predictable, while others exhibit greater volatility.

In four of five folds, evaluation error is lower than training error, indicating that the model generalized well and did not overfit. Early stopping triggered between 100 and 999 iterations depending on the fold's difficulty. Only Fold 5 showed mild overfitting, which motivated the reduction of model complexity (max_depth=3, n_estimators=150) for the final model.

Compared to the linear regression baseline (RMSE = 7.00 Wh), XGBoost achieved a ~19% reduction in RMSE, confirming that nonlinear feature interactions, such as hour × season or dayofweek × month, carry predictive signal beyond what a linear model can capture.

Final XGBoost Model Training

The final XGBoost model is trained on the complete training set using n_estimators=400, max_depth=3, and learning_rate=0.01. The training RMSE decreased smoothly from 13.45 Wh at iteration 0 to 7.98 Wh at iteration 400, indicating stable convergence without any sign of instability. The model was still improving slightly at the final iteration, confirming that 400 trees is a reasonable choice. No early stopping was triggered, meaning the model benefited from all trees in its budget.



Compared to the linear regression baseline, the final XGBoost model achieves a lower training RMSE, and the cross-validation results (~5.70 Wh on evaluation sets) confirm that this improvement generalizes to unseen data.

File Execution Order

1. imports
2. data_ingestion
3. feature-engineering
4. data-analysis
5. data_processing
6. model_building